PROJETO Próxima Etapa

Testes e Qualidade de Software (DevOps)

INTEGRANTES DO PROJETO e RA'S

Bruno De Brito Faccio	-	25028271
Fátima Eduarda Gomes Balbino	-	25028242
José Antônio Da Silva Aguiar	-	25028150
Luiza Sardinha Alde	-	25028264

São Paulo

2025

Sumário

1.1. Apresentar 4 testes unitários. **8**
1.2. Apresentar 2 testes de integração **8**
1.3. Apresentar um teste de usuário (SISTEMA). **9**

INTRODUÇÃO

O projeto Próxima Etapa – App do Aluno propõe o desenvolvimento de um aplicativo Android integrado a uma API e a um banco de dados, com o objetivo de facilitar o acesso dos estudantes às informações de sua jornada educacional. O aplicativo permitirá consultar cursos, agenda, presença por QR Code, certificados, card digital, testes de perfil, mensagens e notificações. O projeto será direcionado aos alunos da Próxima Etapa, mantendo foco em segurança, acessibilidade e privacidade dos dados.

1. Teste Unitário De Software

- Função: A função validar() é responsável por verificar se os dados informados pelo usuário no formulário de cadastro estão corretos antes de permitir que o cadastro seja realizado.

```
private String validar(String nome, String email, String telefone,
String senha, String confirma) {

if (nome.length() < 3) {
return getString(R.string.erro_nome);
}

if (!Patterns.EMAIL_ADDRESS.matcher(email).matches()) {
return getString(R.string.erro_email);
}

if (telefone.length() < 10 || telefone.length() > 11) {
return getString(R.string.erro_telefone);
}

if (senha.length() < 6) {
return getString(R.string.erro_senha_curta);
}

if (!senha.equals(confirma)) {
return getString(R.string.erro_senhas_diferentes);
}

return null;
}
```

1. Cadastro válido	Nome: Bruno Castro; E-mail: bruno@email.com; Telefone: 11999999999; Senha: 123456; Confirmação: 123456	Retornar null, indicando que todos os dados são válidos.	Aprovado
2. Nome inválido	Nome: Br; demais dados válidos.	Retornar erro de nome, pois possui menos de 3 caracteres.	Aprovado
3. E-mail inválido	E-mail: brunoemail.com; demais dados válidos.	Retornar erro de e-mail, pois o formato é inválido.	Aprovado
4. Telefone inválido	Telefone: 12345; demais dados válidos.	Retornar erro de telefone, pois possui menos de 10 dígitos.	Aprovado
5. Senha curta	Senha: 123; Confirmação: 123; demais dados válidos.	Retornar erro de senha curta, pois possui menos de 6 caracteres.	Aprovado
6. Senhas diferentes	Senha: 123456; Confirmação: 654321; demais dados válidos.	Retornar erro de senhas diferentes.	Aprovado

- Função: A função abrirCursos() é responsável por abrir a tela de cursos do aplicativo, permitindo também enviar um termo de pesquisa para que a tela apresente os cursos relacionados à busca.

```
public void abrirCursos(String busca) {
   ignorarSelecao = true;
   navegacao.setSelectedItemId(R.id.nav_cursos);
   ignorarSelecao = false;
   trocar(CursosFragment.novo(busca));
}
```

1. Pesquisa preenchida	Busca: "Programação"	Abrir a tela de cursos e encaminhar o termo de pesquisa corretamente.	Aprovado
2. Pesquisa vazia	Busca: ""	Abrir a tela de cursos normalmente, sem termo de pesquisa.	Aprovado
3. Pesquisa específica	Busca: "Análise e Desenvolvimento de Sistemas"	Abrir a tela de cursos e encaminhar o nome completo para pesquisa.	Aprovado
4. Caracteres especiais	Busca: "Programação Java & Python"	Abrir a tela de cursos e encaminhar o texto preservando os caracteres especiais.	Aprovado

- Função: A função mostrarAgenda() é responsável por organizar e exibir os próximos compromissos do aluno na tela inicial do aplicativo.

```
private static final int MAX_ITENS_AGENDA = 3;

private void mostrarAgenda(List<Encontro> encontros, boolean definitivo)
{
   List<Encontro> primeiros = encontros.size() > MAX_ITENS_AGENDA
         ? encontros.subList(0, MAX_ITENS_AGENDA)
         : encontros;

   adaptador.submeter(primeiros);

   tvAgendaVazia.setVisibility(
         primeiros.isEmpty() && definitivo ? View.VISIBLE : View.GONE
   );
}
```

1. Mais de 3 eventos	5 eventos, definitivo = true	Exibiu somente os 3 primeiros eventos.	Aprovado

2. Exatamente 3 eventos	3 eventos, definitivo = true	Exibiu os 3 eventos.	Aprovado
3. Apenas 1 evento	1 evento, definitivo = true	Exibiu somente 1 evento.	Aprovado
4. Agenda vazia definitiva	Lista vazia, definitivo = true	Exibiu a mensagem de agenda vazia.	Aprovado
5. Agenda vazia não definitiva	Lista vazia, definitivo = false	Não exibiu a mensagem de agenda vazia.	Aprovado

- Função: A função salvar() é responsável por validar os dados informados pelo usuário e atualizar as informações do perfil.

```
private void salvar(Button botao) {
    if (salvando) {
        return;
    }

    String nome = etNome.getText().toString().trim().replaceAll("\\s+", " ");
    String telefone = etTelefone.getText().toString().replaceAll("\\D", "");

    if (nome.length() < 3) {
        mostrarErro(getString(R.string.erro_nome));
        return;
    }

    if (telefone.length() < 10 || telefone.length() > 11) {
        mostrarErro(getString(R.string.erro_telefone));
        return;
    }

    salvando = true;
    botao.setEnabled(false);
    tvErro.setVisibility(View.GONE);

    repositorio.atualizar(nome, telefone, (sucesso, falha) -> {
        salvando = false;
```

```
        if (!isAdded()) {
            return;
        }

        if (sucesso) {
            getParentFragmentManager().setFragmentResult(RESULTADO, new
Bundle());
            dismiss();
            return;
        }

        if (falha.isSessaoExpirada()) {
            dismiss();
            ((MainActivity) requireActivity()).encerrarSessao(true);
            return;
        }

        botao.setEnabled(true);
        mostrarErro(falha.getMensagem());
    });
}
```

1. Nome com 2 caracteres (Jo)	Rejeitar nome	Aprovado
2. Nome válido (Bruno)	Aceitar nome	Aprovado
3. Telefone com 9 dígitos	Rejeitar telefone	Aprovado
4. Telefone com 12 dígitos	Rejeitar telefone	Aprovado
5. Telefone com 10 dígitos	Aceitar telefone	Aprovado
6. Telefone com 11 dígitos	Aceitar telefone	Aprovado
7. Telefone formatado (11) 91234-5678	Remover caracteres especiais	Aprovado
8. Nome com espaços extras	Normalizar para Bruno Castro	Aprovado

2. Teste De Integração De Software

- As funções que irão ser analisadas em conjunto são carregar() e inscrever().

Função carregar(): Responsável por buscar os cursos disponíveis na API e no banco de dados local, atualizar as informações e disponibilizar a lista de cursos para que o aluno possa visualizá-los no aplicativo.

Função inscrever(): Responsável por realizar a inscrição do aluno em um curso selecionado, enviando o identificador do curso para a API e verificando se a operação foi concluída com sucesso.

Funcionamento em conjunto: As duas funções deverão trabalhar integradas para permitir que o aluno visualize os cursos disponíveis e realize sua inscrição. A função carregar() disponibilizará a lista de cursos, enquanto a função inscrever() utilizará o ID do curso selecionado para enviar a solicitação de inscrição à API, verificando se a operação foi realizada corretamente.

Inicialmente, foi verificado se a função carregar() conseguiu buscar os cursos disponíveis na API e no banco de dados local, disponibilizando as informações para o aplicativo. Em seguida, foi simulada a seleção de um curso pelo aluno, verificando se o sistema recuperou corretamente seu identificador (ID). Posteriormente, foi analisado se a função inscrever() recebeu esse identificador e enviou a solicitação de inscrição à API.

Também foram avaliadas situações de erro, como a tentativa de inscrição em um curso com ID inválido, a indisponibilidade da API e possíveis falhas de conexão. Dessa forma, foi possível analisar a comunicação entre as duas funções e verificar se o sistema apresentou os resultados esperados em cada cenário.

Cenários colocados e resultado esperado:

- Carregar os cursos e selecionar um deles. / O curso deverá ser apresentado corretamente e possuir um ID válido.

- Carregar os cursos e realizar a inscrição em um curso existente. / A API deverá receber o ID correto e confirmar a inscrição.

- Tentar realizar a inscrição utilizando um ID inexistente. / O sistema deverá identificar a solicitação inválida e retornar uma falha.

- Tentar carregar os cursos sem conexão com a API. / O sistema deverá manter os dados locais disponíveis e informar a falha de atualização.

- Carregar os cursos e tentar realizar a inscrição sem conexão com a API. / O carregamento local deverá funcionar, mas a inscrição deverá retornar uma falha de conexão.

Cenário	Resultado previsto	Status
Carregar os cursos e selecionar um deles.	O sistema deverá apresentar os cursos e disponibilizar um ID válido.	Aprovado
Realizar a inscrição em um curso existente.	A API deverá receber o ID e retornar a confirmação da inscrição.	Aprovado
Realizar a inscrição utilizando um ID inexistente.	O sistema deverá identificar a falha e retornar inscrição não concluída.	Aprovado
Carregar os cursos sem conexão com a API.	O sistema deverá disponibilizar os cursos locais e identificar a falha de atualização.	Aprovado
Carregar os cursos e tentar realizar a inscrição sem conexão.	Os cursos locais deverão continuar disponíveis, enquanto a inscrição deverá retornar falha.	Aprovado

- As próximas a serem utilizadas serão carregar() e atualizar().

Função carregar(): Responsável por buscar os dados do perfil do aluno na API, recuperar informações como nome, e-mail e telefone e disponibilizá-las para visualização no aplicativo.

Função atualizar(): Responsável por enviar as alterações realizadas nos dados do perfil do aluno para a API, solicitando a atualização das informações e verificando se a operação foi concluída com sucesso.

Funcionamento em conjunto: As duas funções deverão trabalhar integradas para permitir que o aluno consulte e atualize seus dados pessoais. Inicialmente, a função carregar() buscará as informações do perfil na API e as disponibilizará para visualização. Em seguida, após as alterações realizadas pelo aluno, a função atualizar() enviará os novos dados para a API, verificando se as informações foram atualizadas corretamente.

O teste de integração teve como objetivo verificar se as funções carregar() e atualizar() trabalharam corretamente em conjunto, garantindo a comunicação adequada entre o

aplicativo e a API durante o processo de consulta e atualização dos dados do perfil do aluno.

Inicialmente, foi verificado se a função carregar() conseguiu realizar a solicitação à API e recuperar as informações cadastradas do aluno, como nome, e-mail e telefone. Após o recebimento desses dados, foi analisado se as informações foram processadas e disponibilizadas corretamente na interface do aplicativo, permitindo sua visualização e edição.

Posteriormente, foram simuladas alterações nos dados pessoais do aluno, como a modificação do nome e do telefone. Essas informações foram encaminhadas por meio da função atualizar(), responsável por enviar uma requisição à API solicitando a atualização dos dados cadastrados. Durante essa etapa, foi verificado se os dados modificados foram enviados corretamente e se a API retornou uma resposta confirmando a operação.

Além disso, foram consideradas situações que poderiam comprometer o funcionamento integrado das duas funções, como falhas de conexão com a API, envio de informações inválidas e erros durante o processamento da atualização.

Por fim, o teste buscou verificar se o fluxo completo de consulta, edição e atualização dos dados pessoais ocorreu de maneira adequada, garantindo que as informações apresentadas no aplicativo correspondessem aos dados atualizados no sistema. Dessa forma, foi possível avaliar a integração entre as funcionalidades e identificar possíveis falhas na comunicação e no processamento das informações.

Cenários colocados e resultados esperados:

- Carregar os dados do perfil de um aluno existente. | O sistema deverá recuperar corretamente as informações cadastradas, como nome, e-mail e telefone.

- Carregar os dados do perfil e alterar o nome do aluno. | O sistema deverá enviar o novo nome à API e confirmar a atualização.

- Carregar os dados do perfil e alterar o telefone do aluno. | O sistema deverá enviar o novo telefone à API e atualizar a informação cadastrada.

- Tentar atualizar o perfil com informações inválidas ou campos obrigatórios vazios. | O sistema deverá identificar os dados inválidos e impedir a atualização.

- Tentar carregar e atualizar os dados sem conexão com a API. | O sistema deverá identificar a falha de conexão, informar o erro e evitar que a atualização seja considerada concluída.

- Carregar o perfil após uma atualização bem-sucedida. | O sistema deverá recuperar os dados atualizados e apresentar as novas informações do aluno.

Cenário de teste	Resultado obtido	Status
Carregar os dados do perfil de um aluno existente.	Os dados do perfil foram recuperados corretamente.	Aprovado
Carregar o perfil e alterar o nome do aluno.	O novo nome foi enviado à API e a atualização foi confirmada.	Aprovado
Carregar o perfil e alterar o telefone do aluno.	O novo telefone foi enviado à API e atualizado.	Aprovado
Atualizar o perfil com informações inválidas.	O sistema identificou os dados inválidos e impediu a atualização.	Aprovado
Carregar e atualizar os dados sem conexão com a API.	O sistema identificou a falha de conexão e não confirmou a atualização.	Aprovado
Carregar o perfil após uma atualização bem-sucedida.	Os dados atualizados foram recuperados corretamente.	Aprovado

Os testes de integração tiveram como objetivo verificar o funcionamento conjunto das funções carregar(), inscrever() e atualizar(). Foram avaliados o carregamento de cursos, a realização de inscrições, a consulta e atualização dos dados pessoais, além do tratamento de possíveis falhas. Dessa forma, foi possível identificar a importância da comunicação entre as funcionalidades para garantir o funcionamento correto do aplicativo e a consistência das informações.

3. Teste Sistema De Software

O que irá ser analisado: Os testes de sistema tiveram como objetivo avaliar o funcionamento geral do aplicativo Próxima Etapa, verificando se todas as funcionalidades operaram corretamente quando utilizadas de maneira integrada. Durante a análise, foram avaliados os principais recursos disponíveis, como login, cadastro, consulta de cursos, agenda acadêmica, edição de perfil, registro de presença por QR Code e emissão de certificados. Também foram verificadas a navegação entre as telas, a comunicação com a API, a apresentação das informações e o tratamento de possíveis erros. Dessa forma, buscou-se identificar falhas que pudessem comprometer a experiência do usuário e verificar se o aplicativo atendeu aos requisitos funcionais e não funcionais estabelecidos no projeto.

No sistema Próxima Etapa, o teste de sistema consiste em avaliar o aplicativo completo, simulando a utilização por um aluno, desde o momento em que ele realiza o acesso até a utilização dos recursos disponíveis.

A análise foi organizada da seguinte maneira:

1. Acesso ao aplicativo: foi verificado o fluxo de entrada por meio do login, incluindo a validação do e-mail e da senha, além da possibilidade de acesso à tela de cadastro.

2. Navegação: foi analisada a movimentação entre as telas principais, verificando o funcionamento dos menus, botões e atalhos da tela inicial.

3. Cursos: foi considerado o carregamento da lista de cursos, a aplicação dos filtros, a pesquisa e a visualização dos detalhes.

4. Agenda: foi analisada a consulta dos compromissos e encontros acadêmicos apresentados ao aluno.

5. Perfil: foi considerado o acesso às informações pessoais e a possibilidade de edição dos dados cadastrados.

6. Presença: foi analisado o fluxo de consulta dos cursos e de utilização do leitor de QR Code para registrar a presença.

7. Certificados: foi considerado o carregamento dos certificados e a geração dos documentos em PDF.

8. Outras funcionalidades: também foram analisados os recursos de card digital, testes de perfil, comunicados, mensagens e privacidade.

O que foi verificado no código: o projeto possui telas específicas para essas funcionalidades, além de comunicação com repositórios responsáveis pelo acesso aos dados e à API.

Funcionalidade	Resultado esperado	Status
Login e autenticação	Acesso realizado corretamente	Aprovado
Cadastro de aluno	Cadastro realizado com sucesso	Aprovado
Tela inicial e navegação	Telas acessadas corretamente	Aprovado
Consulta de cursos	Cursos exibidos corretamente	Aprovado
Agenda	Eventos apresentados corretamente	Aprovado
Perfil do aluno	Dados exibidos corretamente	Aprovado
Presença por QR Code	Leitura e registro realizados	Aprovado
Certificados	Certificados apresentados corretamente	Aprovado
Teste de perfil	Teste concluído e resultado exibido	Aprovado
Card digital	Cartão apresentado corretamente	Aprovado
Comunicados e mensagens	Informações exibidas corretamente	Aprovado
Privacidade	Opções de privacidade acessíveis	Aprovado

Concluiu-se que os testes de sistema permitiram avaliar as principais funcionalidades do aplicativo Próxima Etapa, verificando os resultados esperados para cada recurso. De modo geral, os cenários avaliados apresentaram resultados satisfatórios, indicando que o sistema atende aos requisitos propostos. Dessa forma, considera-se que o aplicativo apresentou um desempenho adequado dentro dos critérios definidos para a avaliação.